

Multi-Tenant Domain-Based Enterprise RAG Platform

Complete Developer Documentation

Next.js 15 · FastAPI · PostgreSQL + pgvector · Redis · Docker

Version 2.0 · September 2026 · Development Environment

1. Project Overview

The Multi-Tenant Domain-Based Enterprise RAG Platform is a B2B SaaS product that delivers secure, AI-powered knowledge management across independent business departments for multiple companies simultaneously.

Each company (tenant) is fully isolated from every other company. Within a company, each department (domain — HR, Finance, IT, Legal, Operations) maintains its own knowledge base, AI agent, roles, and document workflows — all powered by shared infrastructure.

Attribute	Value
Platform type	B2B SaaS — multi-tenant
Deployment target	Local Docker (dev) → cloud (prod)
Frontend	Next.js 15 (App Router) + TypeScript
Backend	FastAPI (Python 3.12)
Primary database	PostgreSQL 16 + pgvector extension
Vector search	pgvector — HNSW index, cosine similarity
Cache + queue broker	Redis 7
Async workers	Celery
File storage	MinIO (S3-compatible, local dev)
Auth	Keycloak + JWT with RBAC claims
AI framework	LangChain + LiteLLM + OpenAI GPT-4o
Audit logs	ClickHouse
Containerisation	Docker Compose (dev)

1.1 Tenant vs Domain — Core Concept

These two concepts are completely different and must never be confused:

	Tenant	Domain
What it is	A company / organisation	A department / business function
Created by	Platform super admin	Tenant admin
Isolation	Full — RLS + separate namespaces	RBAC-enforced within the tenant
Examples	Acme Corp, Globex Inc	HR, Finance, IT, Legal, Ops
Has its own	DB rows, S3 prefix, vector namespace	Knowledge base, AI agent, roles
Billed at	Tenant level	Not billed separately

2. Technology Stack & Reasoning

2.1 Frontend Libraries

Library	Version	Purpose	Why chosen
Next.js	15.x	Frontend framework	App Router, middleware for tenant auth, BFF API routes, SSR
React	18.x	UI runtime	Server + client components, Suspense streaming
TypeScript	5.x	Type safety	Catch tenant/domain ID mismatches at compile time
Tailwind CSS	3.x	Styling	Utility-first, no runtime, easy dark mode
shadcn/ui	latest	Component system	Accessible, Radix-based, copy-owned components
Zustand	4.x	Global state	Tenant context, domain context, auth state
TanStack Query	5.x	Server state	Caching, background refetch, optimistic updates
TanStack Table	8.x	Data tables	Sorting, filtering, pagination for doc/user lists
React Hook Form	7.x	Forms	Performant forms with minimal re-renders
Zod	3.x	Schema validation	Shared schemas between frontend and backend
Axios	1.x	HTTP client	Interceptors for JWT injection and refresh
next-auth	5.x	Auth sessions	Keycloak OIDC provider, session persistence
Framer Motion	11.x	Animations	Page transitions, chat message animations
Recharts	2.x	Charts	Usage dashboards, token consumption graphs
React Dropzone	14.x	File upload	Drag-and-drop document upload UI
React Markdown	9.x	Markdown render	Stream and render AI responses
date-fns	3.x	Date utils	Format audit timestamps, relative dates
lucide-react	0.383.x	Icons	Consistent icon system

2.2 Backend Libraries (FastAPI / Python)

Library	Version	Purpose	Why chosen
FastAPI	0.111.x	Web framework	Async-native, auto OpenAPI docs, Pydantic integration
Pydantic v2	2.x	Validation + settings	Request/response schemas, env config management
SQLAlchemy	2.x	ORM	Async sessions, declarative models, migration support
Alembic	1.x	DB migrations	Version-controlled schema changes
asyncpg	0.29.x	PG driver	Fastest async PostgreSQL driver
pgvector	0.3.x	Vector ORM type	SQLAlchemy Vector type for pgvector columns
python-jose	3.x	JWT	Token encode/decode/verify
passlib + bcrypt	1.x	Password hashing	Industry-standard bcrypt hashing
LangChain	0.2.x	RAG pipeline	Document loaders, splitters, retrieval chains
LiteLLM	1.x	LLM gateway	Swap OpenAI / Anthropic / local models per domain
openai	1.x	Embeddings + LLM	text-embedding-3-large + GPT-4o
Unstructured	0.14.x	Doc parsing	PDF, DOCX, PPTX, Excel multi-format extraction
pytesseract	0.3.x	OCR	Scanned image text extraction
pandas	2.x	Structured data	CSV/Excel row serialisation to text
openpyxl	3.x	Excel reading	XLSX sheet parsing
redis	5.x	Cache client	Session cache, rate limit counters
celery	5.x	Task queue	Async doc ingestion and embedding generation
boto3	1.x	S3/MinIO client	File upload/download with presigned URLs
httpx	0.27.x	Async HTTP	Internal service calls
slowapi	0.1.x	Rate limiting	Per-tenant endpoint rate limits
structlog	24.x	Logging	Structured JSON logs for observability
sentry-sdk	2.x	Error tracking	Real-time error alerts per service
pytest + asyncio	8.x / 0.23.x	Testing	Async test support for FastAPI endpoints

2.3 Infrastructure

Technology	Role	Why chosen
PostgreSQL 16	Primary DB + vector store	RLS tenant isolation + pgvector in one service — no separate vector DB needed
pgvector	Vector similarity search	HNSW index, cosine similarity, fully transactional, same DB as relational data
Redis 7	Cache + Celery broker	Fast session storage, rate limit counters, Celery task broker
MinIO	Object storage (local dev)	S3-compatible, runs in Docker, no external dependency during dev
ClickHouse	Audit logs	Columnar, append-only, high-write throughput for event logs
NGINX	Reverse proxy	SSL termination, subdomain routing (tenant-a.localhost), load balancing
Keycloak	Identity provider	SSO, MFA, RBAC token claims, runs in Docker for dev
Docker Compose	Local dev orchestration	One command to start all 12 services

2.4 Why pgvector Instead of a Separate Vector Database

The platform uses pgvector (a PostgreSQL extension) instead of a dedicated vector database like Qdrant or Pinecone. Here is the reasoning:

Factor	pgvector	Dedicated vector DB (Qdrant)
Ops complexity	One database to manage	Two services to run and sync
Transactional consistency	Full ACID — chunk + vector in same transaction	Eventual consistency between systems
Tenant isolation	PostgreSQL RLS enforces it automatically	Requires named collections per tenant
Hybrid search	pgvector + tsvector GIN index in one query	Requires separate systems or workarounds
Cost	Free — same PostgreSQL instance	Extra compute, extra service
Scale ceiling	Excellent to ~5–10M vectors	Better beyond 50M+ vectors
Migration path	Easy to add Qdrant later if needed	Harder to go back

💡 For an early-stage SaaS with tens of thousands to a few million chunks, pgvector is the correct choice. The transactional consistency alone — being able to delete a document and its vectors in one ACID transaction — is a significant operational advantage.

3. System Architecture

3.1 Layer Overview

Layer	Technology	Responsibility
Client	Browser / Mobile	User interface, file upload, AI chat
Reverse proxy	NGINX	SSL, subdomain routing (tenant-a.localhost → tenant context), rate limiting
Frontend	Next.js 15	SSR, middleware auth guard, BFF API routes, domain routing
Core API	FastAPI	Auth, tenant management, RAG engine, document processing, LLM gateway
Async workers	Celery + Redis	Document ingestion, OCR, embedding generation (non-blocking)
Domain services	FastAPI (per domain)	HR, Finance, IT domain-specific business logic and AI agents
Primary database	PostgreSQL + pgvector	All relational data + vector embeddings in one service
Cache	Redis	Session cache, rate limit counters, Celery broker
File storage	MinIO	Raw document files, per-tenant path prefixes
Audit	ClickHouse	Append-only event log for all platform activity
Auth	Keycloak	SSO, MFA, JWT with tenant + domain role claims

3.2 Request Flow — AI Query

Every AI query passes through these stages in order:

Step	Layer	Action
1	Browser	User types query in AI chat — JWT in cookie
2	NGINX	Routes tenant-a.localhost to Next.js, injects X-Tenant header
3	Next.js middleware	Verifies JWT signature and expiry — redirects to login if invalid
4	Next.js BFF route	Extracts tenant_id + domain_id from JWT claims, forwards to FastAPI

Step	Layer	Action
5	FastAPI RBAC	Checks user has the required domain role — returns 403 if not
6	RAG engine	Embeds query using OpenAI text-embedding-3-large (3072 dims)
7	pgvector search	SELECT chunks ORDER BY embedding <=> query_vec WHERE tenant_id=\$t AND domain_id=\$d
8	Full-text search	tsvector GIN index keyword search on same chunks table
9	RRF fusion	Reciprocal Rank Fusion merges vector + keyword results
10	Re-ranking	Cross-encoder selects top-K most relevant chunks
11	Domain AI agent	Builds domain-specific prompt with retrieved context
12	LiteLLM → GPT-4o	Generates answer, streamed via Server-Sent Events
13	Next.js UI	Renders streamed markdown response with source citations
14	ClickHouse	Audit event written: user, tenant, domain, query, timestamp, tokens used

3.3 Multi-Tenant Isolation Strategy

Isolation is enforced at multiple independent layers so that a bug in one layer does not cause data leakage:

Layer	Mechanism	What it prevents
Network	NGINX subdomain routing	Wrong tenant context reaching the application
Application	JWT tenant_id claim checked in middleware	Unauthenticated or wrong-tenant requests
API	FastAPI tenant context injected into every DB session	Application-level cross-tenant queries
Database	PostgreSQL Row-Level Security on all tables	Cross-tenant data access even if app has a bug
Vectors	pgvector queries always include WHERE tenant_id + domain_id	Cross-tenant/domain vector retrieval
Files	MinIO paths prefixed tenant_id/domain_id/	File access across tenant boundaries

4. Multi-Format Document Processing

The platform accepts three categories of input and routes each through an appropriate processing pipeline before generating embeddings stored in PostgreSQL via pgvector.

4.1 Supported Formats

Category	Formats	Processing library
Documents	PDF, DOC, DOCX, TXT, PPT, PPTX	Unstructured.io
Structured data	XLS, XLSX, CSV	pandas + openpyxl
Visual / scanned	JPG, PNG, scanned PDF	Tesseract OCR / AWS Textract

4.2 Processing Pipeline

Stage	Action	Tool
1. Upload	File received by FastAPI, validated, stored to MinIO	FastAPI + boto3
2. Type detection	MIME type + magic bytes identify format	python-magic
3. Celery task	Async ingest task queued — user gets immediate response	Celery + Redis
4a. Doc pipeline	Text + layout extracted (headings, tables, lists)	Unstructured.io
4b. Structured pipeline	Rows serialised to natural language + summary stats	pandas + openpyxl
4c. OCR pipeline	Image pre-processed → Tesseract or Textract extracts text	pytesseract
5. Chunking	Recursive character splitter, 512 tokens, 64 token overlap	LangChain
6. Embedding	OpenAI text-embedding-3-large, 3072 dims, batched	openai SDK
7. Storage	Chunk text + embedding upserted to document_chunks table	pgvector + SQLAlchemy

4.3 OCR Decision Logic

For visual and scanned files, the pipeline applies this logic before choosing an OCR engine:

Condition	Action
PDF with native text layer AND DPI $\geq$ 300	Direct extraction via PyMuPDF — no OCR needed
Standard image or low-quality PDF	Pre-process (deskew, denoise, contrast) → Tesseract 5

Condition	Action
Document with tables, forms, or handwriting	AWS Textract (more accurate for structured layouts)

5. Database Architecture (PostgreSQL + pgvector)

5.1 pgvector Setup

Run once in an Alembic migration before any other schema changes:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

5.2 Core Tables

Table	Purpose	Key columns
tenants	One row per company	id (uuid PK), name, slug, subdomain, plan, status, created_at
domains	Global domain registry (defined by platform)	id (uuid PK), name, slug, description, is_active
tenant_domains	Which domains a tenant has activated	tenant_id (FK), domain_id (FK), config (jsonb), activated_at
users	All platform users across all tenants	id (uuid PK), tenant_id (FK), email, password_hash, full_name, status
user_domain_roles	Per-user domain role assignments	user_id (FK), domain_id (FK), role (enum), granted_by, granted_at
invite_tokens	Pending email invitations	id (uuid PK), tenant_id, email, domain_roles (jsonb), expires_at, used_at
sessions	Active JWT refresh sessions	id (uuid PK), user_id (FK), refresh_token_hash, expires_at, ip_address

5.3 Document & Knowledge Tables

Table	Purpose	Key columns
documents	Uploaded file metadata	id, tenant_id, domain_id, user_id, filename, mime_type, s3_key, status, page_count, created_at
document_chunks	Text chunks + vectors (pgvector)	id, tenant_id, domain_id, document_id, chunk_index, content (TEXT), embedding (vector(3072)), metadata (jsonb), created_at
processing_jobs	Async ingestion job status	id, document_id, status (queued/processing/done/failed), error, started_at, completed_at

5.4 pgvector Column and Index

The `document_chunks` table stores the raw text and its embedding in the same row — no separate vector store needed:

```
-- Add vector column
ALTER TABLE document_chunks ADD COLUMN embedding vector(3072);

-- HNSW index for fast approximate nearest neighbour search
CREATE INDEX ON document_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Full-text search index (BM25-style keyword search)
CREATE INDEX ON document_chunks USING gin(to_tsvector('english', content));
```

5.5 Hybrid Search Query

The retrieval query combines vector similarity and keyword search, scoped by tenant and domain:

```
SELECT
  id, content, metadata,
  1 - (embedding <=> $1::vector) AS vec_score,
  ts_rank(to_tsvector('english', content), query) AS text_score
FROM document_chunks, to_tsquery('english', $4) query
WHERE tenant_id = $2
  AND domain_id = $3
  AND (
    (embedding <=> $1::vector) < 0.4 -- cosine similarity threshold
    OR to_tsvector('english', content) @@ query
  )
ORDER BY (vec_score * 0.7 + text_score * 0.3) DESC
LIMIT 20;
```

5.6 Row-Level Security

RLS ensures tenant isolation at the database engine level, independent of application code:

```
-- Enable on all tenant-scoped tables
ALTER TABLE document_chunks ENABLE ROW LEVEL SECURITY;

-- Policy: each request can only see its own tenant's rows
CREATE POLICY tenant_isolation ON document_chunks
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- FastAPI sets this at the start of every DB session:
await session.execute(
  text("SET LOCAL app.tenant_id = :tid"),
  {"tid": str(tenant_id)}
)
```

5.7 Chat & RAG Tables

Table	Purpose	Key columns
conversations	Chat session per user and domain	id, tenant_id, domain_id, user_id, title, created_at
messages	Individual messages in a conversation	id, conversation_id, role (user/assistant), content, sources (jsonb), token_count, created_at
retrieval_logs	Which chunks were retrieved per query	id, message_id, retrieved_chunks (jsonb), rerank_scores (jsonb)

5.8 Billing & Usage Tables

Table	Purpose	Key columns
subscriptions	Stripe plan per tenant	id, tenant_id, stripe_customer_id, stripe_subscription_id, plan, status
usage_records	Monthly usage metering	id, tenant_id, domain_id, period_start, tokens_used, api_calls, storage_bytes

5.9 Audit Table (ClickHouse)

Column	Type	Description
event_id	UUID	Unique event identifier
tenant_id	UUID	Which tenant performed the action
domain_id	UUID	Which domain (nullable for platform events)
user_id	UUID	Who performed the action
event_type	String	login, query, upload, role_change, delete, etc.
resource_type	String	document, conversation, user, tenant, etc.
resource_id	UUID	ID of the affected resource
ip_address	String	Client IP address
metadata	JSON	Event-specific extra data
created_at	DateTime	UTC timestamp — used as partition key

6. Project Folder Structure

6.1 Monorepo Root

```

platform/
├── apps/
│   ├── web/           # Next.js 15 frontend
│   ├── api/           # FastAPI core platform
│   └── domains/
│       ├── hr/        # HR domain service
│       ├── finance/   # Finance domain service
│       └── it/         # IT domain service
├── packages/
│   ├── rag-engine/    # Shared RAG pipeline (Python)
│   ├── doc-processor/ # Shared document ingestion (Python)
│   ├── auth/          # Shared auth utilities
│   └── types/         # Shared TypeScript types
├── infra/
│   ├── nginx/         # nginx.conf, domain configs
│   └── scripts/        # DB seed, dev utilities
├── docker-compose.yml # Full local dev stack
└── .github/workflows/ # CI/CD pipelines
  
```

6.2 Next.js Frontend (apps/web)

```

apps/web/
├── app/
│   ├── (auth)/        # No tenant layout group
│   │   ├── login/page.tsx
│   │   ├── register/page.tsx
│   │   ├── forgot-password/page.tsx
│   │   ├── reset-password/page.tsx
│   │   └── invite/[token]/page.tsx
│   ├── (platform)/    # Super admin layout group
│   │   ├── layout.tsx  # Platform admin shell
│   │   └── platform/
│   │       ├── dashboard/page.tsx
│   │       ├── tenants/
│   │       │   ├── page.tsx  # Tenant list
│   │       │   └── [id]/page.tsx # Tenant detail
│   │       ├── billing/page.tsx
│   │       ├── users/page.tsx
│   │       ├── models/page.tsx # LLM config
│   │       ├── domains/page.tsx # Domain registry
│   │       ├── audit/page.tsx
│   │       ├── analytics/page.tsx
│   │       └── settings/page.tsx
│   └── [tenant]/       # Tenant-scoped routes
│       ├── layout.tsx  # Tenant shell + sidebar
│       ├── dashboard/page.tsx
│       ├── domains/page.tsx
│       ├── users/
│       │   ├── page.tsx
│       │   └── [id]/roles/page.tsx
│       └── invitations/page.tsx
  
```

```

├── billing/page.tsx
├── audit/page.tsx
├── settings/
│   ├── page.tsx
│   └── notifications/page.tsx
├── [domain]/ # Domain-scoped routes
│   ├── layout.tsx # Domain shell
│   ├── page.tsx # Domain home
│   ├── chat/
│   │   ├── page.tsx
│   │   ├── history/page.tsx
│   │   └── [id]/page.tsx
│   ├── knowledge/
│   │   ├── page.tsx
│   │   ├── upload/page.tsx
│   │   └── [id]/page.tsx
│   ├── search/page.tsx
│   ├── users/page.tsx
│   └── settings/page.tsx
├── account/
│   ├── profile/page.tsx
│   ├── domains/page.tsx
│   ├── notifications/page.tsx
│   └── activity/page.tsx
├── 403/page.tsx
├── 404/page.tsx
├── layout.tsx # Root layout
├── components/
│   ├── ui/ # shadcn/ui base components
│   ├── layout/ # Shell, sidebar, topbar, breadcrumb
│   ├── auth/ # Login form, invite form, MFA
│   ├── chat/ # Chat window, message bubbles, sources
│   ├── knowledge/ # Document list, upload zone, viewer
│   ├── domain/ # Domain card, domain switcher
│   └── platform/ # Tenant tables, usage charts
├── lib/
│   ├── api.ts # Axios instance + JWT interceptor
│   ├── auth.ts # next-auth config (Keycloak provider)
│   └── utils.ts
├── hooks/ # Custom React hooks
├── stores/ # Zustand stores
├── types/ # TypeScript interfaces
├── middleware.ts # JWT + tenant + domain guard
├── next.config.ts
└── tailwind.config.ts

```

6.3 FastAPI Core (apps/api)

```

apps/api/
├── app/
│   ├── main.py # FastAPI app factory, lifespan
│   ├── config.py # Pydantic settings from env
│   ├── database.py # Async SQLAlchemy engine + session
│   ├── dependencies.py # Shared FastAPI dependencies
│   ├── middleware/
│   └── tenant.py # Extract + inject tenant context

```

```

├── rbac.py           # Role + permission enforcement
├── rate_limit.py    # slowapi per-tenant limits
├── routers/
│   ├── auth.py      # /auth/* — login, refresh, invite
│   ├── tenants.py   # /tenants/*
│   ├── users.py     # /users/*
│   ├── domains.py   # /domains/*
│   ├── documents.py # /documents/* — upload, status
│   ├── chat.py      # /chat/* — query (SSE), history
│   ├── search.py    # /search/*
│   ├── billing.py   # /billing/* — Stripe
│   └── audit.py     # /audit/*
├── services/
│   ├── auth_service.py
│   ├── tenant_service.py
│   ├── user_service.py
│   ├── document_service.py
│   ├── embedding_service.py # OpenAI embedding calls
│   ├── rag_service.py      # Hybrid retrieval + rerank
│   └── audit_service.py    # ClickHouse event writes
├── models/                # SQLAlchemy ORM models
│   ├── tenant.py
│   ├── user.py
│   ├── document.py
│   └── chunk.py          # Includes vector(3072) column
├── schemas/              # Pydantic request/response schemas
├── tasks/                # Celery task definitions
│   ├── celery_app.py    # Celery app + Redis config
│   ├── ingest.py        # Doc processing tasks
│   └── embed.py         # Embedding generation tasks
├── utils/
├── tests/
│   ├── conftest.py
│   ├── test_auth.py
│   ├── test_rag.py
│   └── test_documents.py
├── alembic/              # DB migrations
│   ├── env.py
│   └── versions/
│       ├── 001_initial_schema.py
│       ├── 002_add_pgvector.py # CREATE EXTENSION + vector column
│       └── 003_add_hnsw_index.py
├── requirements.txt
└── Dockerfile

```

6.4 Domain Services (apps/domains/hr as example)

```

apps/domains/hr/
├── app/
│   ├── main.py      # Domain FastAPI app
│   └── routers/
│       ├── policies.py # HR policy CRUD
│       ├── handbook.py # Employee handbook
│       └── leave.py    # Leave information
├── services/
└── hr_rag_service.py # HR-specific retrieval + prompts

```

```
| | | hr_agent.py      # LangChain HR AI agent
| | | schemas/
| | | requirements.txt
| | | Dockerfile
```

7. Docker Compose — Local Development

7.1 Services

Service	Image	Port	Role
nginx	nginx:alpine	80	Reverse proxy, subdomain routing
web	node:20-alpine	3000	Next.js 15 frontend
api	python:3.12-slim	8000	FastAPI core platform
worker	python:3.12-slim	—	Celery async workers
hr	python:3.12-slim	8001	HR domain service
finance	python:3.12-slim	8002	Finance domain service
it	python:3.12-slim	8003	IT domain service
postgres	postgres:16	5432	PostgreSQL + pgvector
redis	redis:7-alpine	6379	Cache + Celery broker
minio	minio/minio	9000, 9001	File storage (S3-compatible)
clickhouse	clickhouse/clickhouse-server	8123	Audit log database
keycloak	quay.io/keycloak/keycloak	8080	Identity provider (dev)

7.2 docker-compose.yml

```
version: '3.9'

services:
  nginx:
    image: nginx:alpine
    ports: ['80:80']
    volumes: ['./infra/nginx:/etc/nginx/conf.d:ro']
    depends_on: [web, api]

  web:
    build: ./apps/web
    ports: ['3000:3000']
    environment:
      NEXTAUTH_URL: http://localhost:3000
      NEXTAUTH_SECRET: dev-secret-change-in-prod
      NEXT_PUBLIC_API_URL: http://api:8000
      KEYCLOAK_CLIENT_ID: platform-web
      KEYCLOAK_CLIENT_SECRET: dev-secret
      KEYCLOAK_ISSUER: http://keycloak:8080/realms/platform
    depends_on: [api, keycloak]

  api:
```

```

build: ./apps/api
ports: ['8000:8000']
environment:
  DATABASE_URL: postgresql+asyncpg://user:pass@postgres:5432/platform
  REDIS_URL: redis://redis:6379/0
  S3_ENDPOINT: http://minio:9000
  S3_ACCESS_KEY: minioadmin
  S3_SECRET_KEY: minioadmin
  OPENAI_API_KEY: ${OPENAI_API_KEY}
  JWT_SECRET: dev-jwt-secret-change-in-prod
  CLICKHOUSE_URL: http://clickhouse:8123
depends_on: [postgres, redis, minio, clickhouse]

```

```

worker:
  build: ./apps/api
  command: celery -A app.tasks.celery_app worker -Q ingest,embed -c 4
  environment: *api-env # same env as api service
  depends_on: [redis, postgres]

```

```

postgres:
  image: postgres:16
  ports: ['5432:5432']
  volumes: ['pg_data:/var/lib/postgresql/data']
  environment:
    POSTGRES_DB: platform
    POSTGRES_USER: user
    POSTGRES_PASSWORD: pass

```

```

redis:
  image: redis:7-alpine
  ports: ['6379:6379']

```

```

minio:
  image: minio/minio
  command: server /data --console-address ':9001'
  ports: ['9000:9000', '9001:9001']
  volumes: ['minio_data:/data']
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin

```

```

clickhouse:
  image: clickhouse/clickhouse-server
  ports: ['8123:8123']
  volumes: ['ch_data:/var/lib/clickhouse']

```

```

keycloak:
  image: quay.io/keycloak/keycloak:24
  command: start-dev
  ports: ['8080:8080']
  environment:
    KEYCLOAK_ADMIN: admin
    KEYCLOAK_ADMIN_PASSWORD: admin

```

```

volumes:
  pg_data: {}

```

```
minio_data: {}  
ch_data: {}
```

8. Development Phases

The platform is built in 6 sequential phases. Each phase delivers working, testable functionality.

Technology Stack

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui

Backend

- FastAPI
- Python
- SQLAlchemy
- Alembic
- Pydantic

Database

- PostgreSQL
- pgvector

Authentication

- Keycloak
- JWT
- Refresh Token / Session Management

AI / RAG

- LiteLLM
- LLM Providers
- Embedding Model
- Hybrid RAG
- Cross-Encoder Re-ranking

Document Processing

- MinIO
- Unstructured
- Tesseract OCR
- pandas
- openpyxl
- Celery
- Redis

Infrastructure

- Docker Compose
- Git
- GitHub

PHASE 1 — CORE FOUNDATION

Week 1

1.1 Monorepo Setup

Description:

Create the project monorepo with a clean structure for frontend, backend, AI, workers, packages, infrastructure, documentation, and GitHub configuration.

Output:

- Git repository
- Monorepo structure
- Development configuration
- Environment variable templates

1.2 Docker Compose Infrastructure

Description:

Create a local development environment using Docker Compose for the required infrastructure services.

Services:

- PostgreSQL
- Redis
- MinIO
- Keycloak
- LiteLLM
- FastAPI
- Celery Worker
- Next.js

Output:

- docker-compose.yml
- All core services running locally

1.3 PostgreSQL Database Schema

Description:

Design the single PostgreSQL database for the complete multi-tenant platform.

Core tables:

- organizations
- domains
- users
- roles
- permissions
- memberships
- invitations
- documents
- document_chunks
- conversations
- messages
- AI configurations
- audit logs

Output:

- SQLAlchemy models
- Database relationships
- Initial database schema

1.4 Alembic Migration System

Description:

Configure Alembic for version-controlled database migrations.

Output:

- Alembic configuration
- Initial migration
- Migration workflow

1.5 pgvector Setup

Description:

Enable PostgreSQL vector search capabilities.

Tasks:

- Enable pgvector extension
- Create vector column
- Configure vector dimensions
- Create HNSW index
- Configure similarity search

Output:

- Vector-enabled PostgreSQL database

1.6 FastAPI Backend Skeleton

Description:

Create the main FastAPI backend architecture.

Tasks:

- Application factory
- API routers
- Middleware
- Dependency injection
- Pydantic settings
- Error handling
- Health endpoint
- API versioning

Output:

- Running FastAPI backend on port 8000

1.7 Authentication Service

Description:

Implement authentication using Keycloak and JWT-based authentication.

Features:

- Login
- Logout
- Access token
- Refresh token
- Token validation
- Password reset
- User session
- Authentication middleware

Output:

- Working authentication API

1.8 Multi-Tenant Organization Service

Description:

Implement organization creation and tenant isolation.

Features:

- Create organization
- Update organization
- Delete/deactivate organization
- Organization status
- Organization configuration
- Tenant identification

Output:

- Organization management API

1.9 Domain / Workspace Service

Description:

Implement organization-level domains/workspaces.

Example:

```
support.ai  
→ acme.support.ai  
→ hr.acme.support.ai  
→ finance.acme.support.ai  
→ it.acme.support.ai
```

Features:

- Domain creation
- Domain mapping
- Domain validation
- Organization-to-domain mapping
- Domain context

Output:

- Domain routing system

1.10 Tenant Resolver Middleware

Description:

Resolve organization and domain information from the incoming hostname.

Example:

```
hr.acme.support.ai
```

becomes:

```
organization = acme  
domain = hr
```

Output:

- Trusted tenant context
- Organization isolation
- Domain isolation

1.11 User and Role Management

Description:

Implement user management and role-based access control.

Roles:

- Super Admin
- Organization Admin
- Domain Admin
- Support Agent
- Customer

Features:

- User creation
- User update
- User deactivation
- Role assignment
- Domain assignment
- Organization membership

Output:

- User and RBAC APIs

1.12 Invitation System

Description:

Allow organization administrators to invite users.

Features:

- Invitation token
- Invitation expiry
- Accept invitation
- Assign organization
- Assign domain
- Assign role

Output:

- Working invitation workflow

1.13 Next.js Application Shell

Description:

Create the shared frontend application.

Tasks:

- App Router
- Layouts
- Middleware
- Authentication guard
- Tenant routing
- Protected routes
- Error pages
- Loading states

Output:

- Working Next.js frontend

1.14 Shared Authentication Pages

Description:

Create reusable authentication pages that work across all organizations.

Pages:

- Login
- Forgot Password
- Reset Password
- Accept Invitation
- Unauthorized
- Session Expired

Example:

acme.support.ai/login

and

matrix.support.ai/login

both use the same login page.

Output:

- Shared multi-tenant authentication UI

PHASE 2 — DOCUMENT MANAGEMENT AND INGESTION

Week 2

2.1 MinIO File Storage

Description:

Configure MinIO for tenant-isolated document storage.

Structure:

organization
→ domain
→ documents

Output:

- File storage service
- Tenant-scoped storage paths
- Secure file access

2.2 Document Upload API

Description:

Create APIs for document uploading and management.

Supported files:

- PDF
- DOCX
- PPTX
- TXT
- XLSX
- CSV
- Images

Output:

- POST /documents
- GET /documents
- GET /documents/:id
- DELETE /documents/:id

2.3 Document Validation

Description:

Validate uploaded documents.

Checks:

- File type
- File size
- File extension
- MIME type
- Organization ownership
- Domain ownership

Output:

- Secure document validation

2.4 Unstructured Document Processing

Description:

Extract structured content from supported documents.

Supported:

- PDF
- DOCX
- PPTX
- TXT

Extract:

- Text
- Headings
- Pages
- Tables
- Metadata

Output:

- Normalized document content

2.5 Excel and CSV Processing

Description:

Process structured data files.

Technology:

- pandas
- openpyxl

Output:

- Serialized table content
- Row-level metadata
- Structured document representation

2.6 OCR Pipeline

Description:

Process scanned documents and images.

Technology:

- Tesseract OCR

Tasks:

- Detect scanned PDFs
- Extract text from images
- Normalize OCR output

Output:

- Searchable OCR content

2.7 Document Chunking

Description:

Split documents into retrieval-friendly chunks.

Configuration:

- Chunk size
- Chunk overlap
- Page metadata
- Section metadata
- Document metadata

Output:

- `document_chunks` records

2.8 Celery Processing Pipeline

Description:

Move heavy document processing into background workers.

Pipeline:

Upload

- Parse
- OCR
- Chunk
- Embed
- Store

Features:

- Retry
- Failure handling
- Processing status
- Background execution

Output:

- Celery workers
- Redis queue

2.9 Embedding Service

Description:

Generate vector embeddings for document chunks.

Tasks:

- Batch embedding
- Error handling
- Retry handling
- Embedding metadata

Output:

- Vector embeddings

2.10 pgvector Storage

Description:

Store document embeddings in PostgreSQL.

Tasks:

- Vector insertion
- HNSW index
- Metadata filtering
- Tenant filtering
- Domain filtering

Output:

- Searchable vector knowledge base

2.11 Knowledge Base UI

Description:

Create the document management interface.

Features:

- Drag and drop upload
- File validation
- Upload progress
- Processing status
- Search documents
- Delete documents
- Document details

Output:

- Knowledge Base pages

PHASE 3 — RAG AND AI ENGINE

Week 3

3.1 Query Processing

Description:

Process user questions before retrieval.

Tasks:

- Query normalization
- Conversation context
- Tenant context
- Domain context

Output:

- RAG-ready query

3.2 Query Embedding

Description:

Generate an embedding for the user's query using the same embedding model used for documents.

Output:

- Query vector

3.3 Vector Search

Description:

Search pgvector for relevant document chunks.

Filters:

- organization_id
- domain_id
- document status
- permissions

Output:

- Top-K vector results

3.4 PostgreSQL Full-Text Search

Description:

Implement keyword-based retrieval using PostgreSQL.

Technology:

- tsvector
- GIN index

Output:

- Keyword search results

3.5 Hybrid Retrieval

Description:

Combine semantic vector search and keyword search.

Flow:

Vector Search
+
Full-Text Search
↓
RRF
↓
Combined Results

Output:

- Hybrid retrieval engine

3.6 Re-Ranking

Description:

Improve retrieval quality using a cross-encoder re-ranker.

Flow:

Top-K Retrieval
↓
Cross Encoder
↓
Relevance Score
↓
Final Context

Output:

- High-quality retrieval results

3.7 AI Configuration

Description:

Create organization/domain-level AI configuration.

Configuration:

- Provider
- Model
- Temperature
- Max tokens
- System prompt
- RAG top-K
- Minimum relevance score

Output:

- AI configuration system

3.8 LiteLLM Gateway

Description:

Provide a unified interface for multiple LLM providers.

Capabilities:

- Provider switching
- Model switching
- Fallback configuration
- Organization-level model configuration

Output:

- LLM abstraction layer

3.9 Domain AI Prompts

Description:

Create domain-specific AI behavior.

Examples:

HR:

Answer questions using HR policies and employee documentation.

Finance:

Answer questions using finance documents and accounting policies.

IT:

Answer questions using technical documentation and troubleshooting guides.

Output:

- Domain-specific prompts

3.10 RAG Answer Generation

Description:

Generate answers using retrieved knowledge.

Flow:

User Question

↓

Retrieval

↓

Re-ranking

↓

Context

↓

LLM

↓

Answer

Output:

- Grounded AI responses

3.11 Source Citations

Description:

Return document sources with every grounded answer.

Metadata:

- Document name
- Page
- Section
- Chunk
- Relevance score

Output:

- Cited AI responses

3.12 SSE Streaming

Description:

Stream AI responses from FastAPI to Next.js.

Output:

- Real-time token streaming
- Streaming Markdown

3.13 Chat History

Description:

Store conversations and messages.

Features:

- Create conversation
- Store messages
- Conversation history
- Pagination
- Delete conversation

Output:

- Persistent chat system

3.14 AI Chat UI

Description:

Create the complete AI assistant interface.

Features:

- Streaming responses
- Markdown
- Source citations
- Copy response
- Regenerate
- Conversation history
- New conversation

Output:

- Complete AI chat page

PHASE 4 — DOMAIN APPLICATIONS AND ADMINISTRATION

Week 4 — Part 1

4.1 Domain Framework

Description:

Create a reusable domain configuration system instead of separate backend applications for every domain.

Domains:

- HR
- Finance
- IT

Output:

- Reusable domain architecture

4.2 HR Application

Features:

- HR policies
- Employee handbook
- Leave policies

- Recruitment documents
- HR knowledge base
- HR AI assistant

Output:

- HR domain application

4.3 Finance Application

Features:

- Finance documents
- Invoice knowledge
- Expense policies
- Accounting documentation
- Finance AI assistant

Output:

- Finance domain application

4.4 IT Application

Features:

- Technical documentation
- API documentation
- Troubleshooting guides
- Incident documentation
- IT AI assistant

Output:

- IT domain application

4.5 Domain RBAC

Description:

Restrict users to their permitted domains.

Example:

HR user → HR RAG ☒

HR user → Finance RAG ☐

Finance user → Finance RAG ☒

Customer → Admin APIs ✕

Output:

- API-level authorization
- UI-level authorization

4.6 Domain Dashboards

Description:

Create dashboards specific to each domain.

Features:

- Document statistics
- AI usage
- Recent documents
- Recent conversations
- Quick actions

Output:

- Domain dashboards

4.7 Domain Pages

Description:

Create domain-specific interfaces.

Pages:

- HR Policies
- Finance Documents
- IT Runbooks
- Knowledge Base
- AI Chat

Output:

- Complete domain UI

4.8 Domain Settings

Description:

Allow authorized administrators to configure AI behavior.

Settings:

- System prompt
- Model
- Temperature
- RAG top-K
- Relevance threshold
- Domain configuration

Output:

- Domain settings interface

PHASE 5 — SAAS ADMINISTRATION AND COMMUNICATION

Week 4 — Part 2

5.1 Organization Administration

Description:

Create organization-level administration.

Features:

- Organization profile
- Users
- Roles
- Domains
- Invitations
- Knowledge base
- AI configuration

Output:

- Organization Admin dashboard

5.2 Platform Administration

Description:

Create Super Admin functionality.

Features:

- Organizations
- Users
- Domains
- System statistics

- Platform configuration
- Organization status

Output:

- Super Admin dashboard

5.3 User Management

Description:

Allow administrators to manage organization users.

Features:

- Add user
- Invite user
- Change role
- Assign domain
- Deactivate user
- View user activity

Output:

- User management UI

5.4 Domain Management

Description:

Allow organization administrators to manage domains/workspaces.

Example:

acme.support.ai

hr.acme.support.ai

finance.acme.support.ai

it.acme.support.ai

Output:

- Domain management UI

5.5 Tenant Branding

Description:

Allow each organization to customize its application appearance.

Configuration:

- Organization name
- Logo
- Favicon
- Primary theme
- Login branding

Output:

- Tenant-specific branding

5.6 Email Service

Description:

Implement transactional email.

Use cases:

- Organization invitations
- Password reset
- Welcome email
- Account notifications

Output:

- Email service

5.7 Notification System

Description:

Create application notifications.

Features:

- Toast notifications
- In-app notifications
- Success/error states
- Document processing notifications
- Invitation notifications

Output:

- Unified notification system

PHASE 6 — TESTING, SECURITY AND FINALIZATION

Final Days

6.1 Authentication Testing

Test:

- Login
- Logout
- Token expiration
- Refresh
- Password reset
- Invitation
- Unauthorized access

6.2 Multi-Tenant Isolation Testing

Test:

Acme user → Acme data	☑
Acme user → Matrix data	✗
Matrix user → Acme data	✗

6.3 Domain Isolation Testing

Test:

HR user → HR documents	☑
HR user → Finance documents	✗
Finance user → Finance	☑
IT user → HR	✗

6.4 RBAC Testing

Test every role against every protected API.

Super Admin
Organization Admin
Domain Admin
Support Agent
Customer

6.5 RAG Testing

Test:

- Retrieval accuracy
- Tenant filtering
- Domain filtering
- Citation accuracy
- No-context questions
- Hallucination handling
- Document deletion
- Updated document retrieval

6.6 API Testing

Test:

- Authentication APIs
- Organization APIs
- Domain APIs
- User APIs
- Document APIs
- RAG APIs
- Chat APIs
- AI configuration APIs

6.7 Frontend Testing

Test:

- Login
- Dashboard
- Domain routing
- Upload
- Knowledge Base
- Chat
- Citations
- Admin pages
- Error states
- Loading states

6.8 Security Configuration

Implement the essential security baseline:

- JWT validation
- Password security

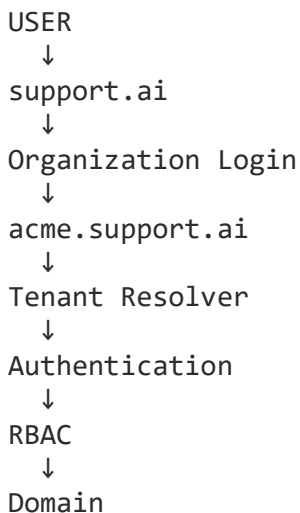
- HttpOnly cookies where applicable
- CORS configuration
- Input validation
- File validation
- Authorization middleware
- Tenant isolation
- Domain isolation
- API authentication
- Secret management
- Rate limiting

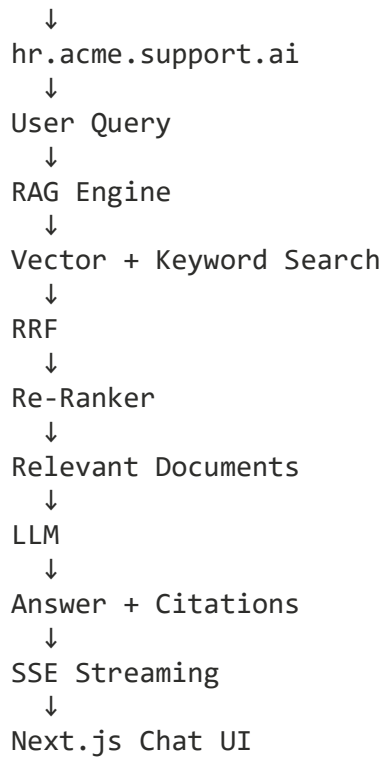
6.9 Documentation

Create:

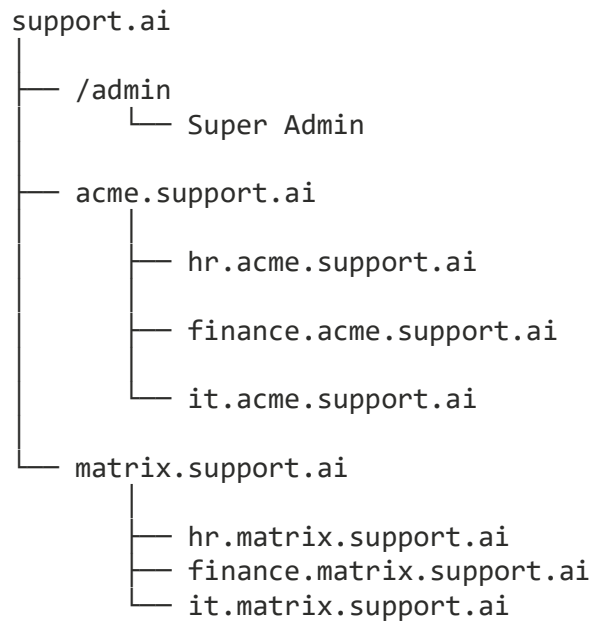
- README.md
- ARCHITECTURE.md
- AUTHENTICATION.md
- MULTI_TENANCY.md
- RBAC.md
- DOMAIN_ROUTING.md
- DOCUMENT_PIPELINE.md
- RAG.md
- API.md
- DATABASE.md
- DEPLOYMENT.md
- CONTRIBUTING.md
- AGENTS.md
- CHANGELOG.md

FINAL PROJECT FLOW



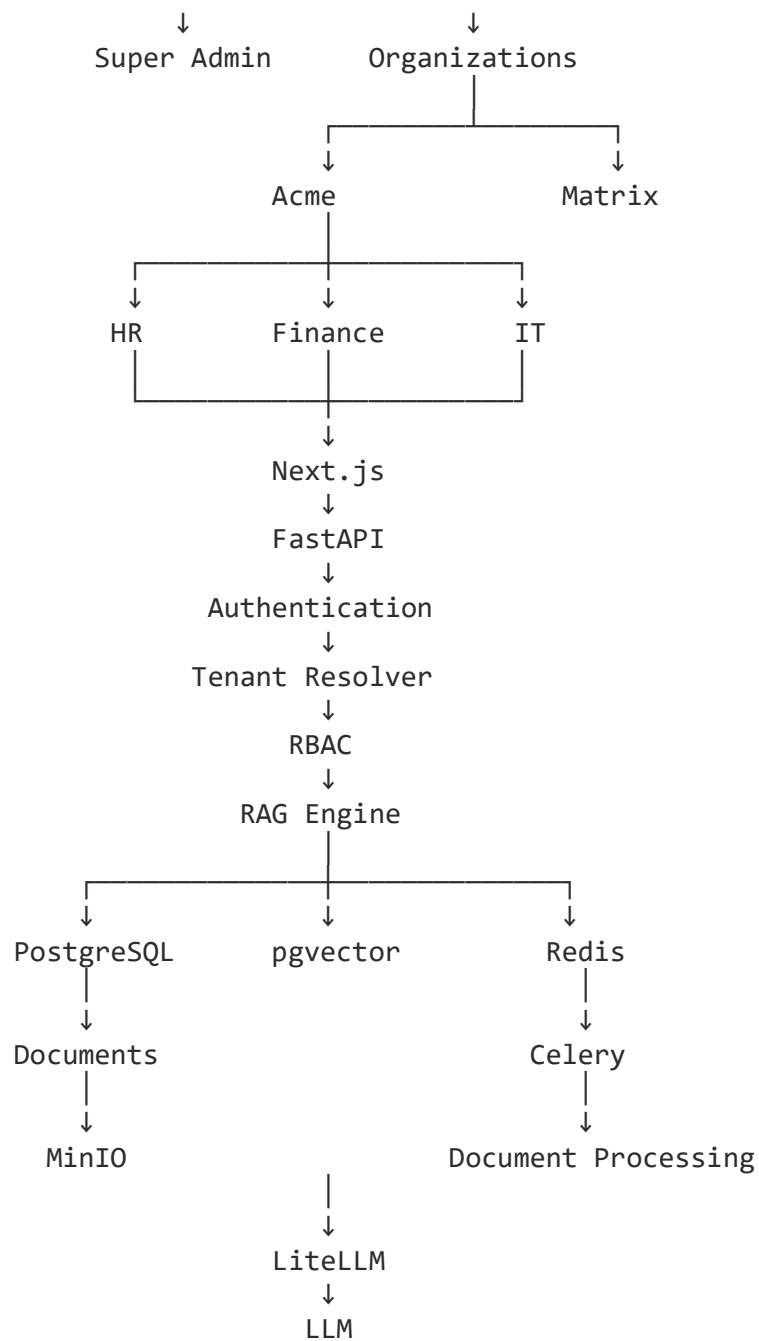


FINAL DOMAIN STRUCTURE



FINAL ARCHITECTURE



**Week 1:**

Multi-tenant foundation + authentication + RBAC + domain routing complete.

Week 2:

Document upload + processing + OCR + embeddings + pgvector complete.

Week 3:

Hybrid RAG + LLM + streaming + citations + chat complete.

Week 4:

HR/Finance/IT + administration + branding + email + testing + documentation complete.

Final MVP

The completed project will demonstrate:

Multi-Tenant SaaS + Organization Management + Domain-Based Routing + Authentication + RBAC + Document Management + OCR + Vector Search + Hybrid RAG + Re-Ranking + LLM Integration + Domain AI + Citations + Streaming Chat + Admin Dashboards + Tenant Branding + Notifications + Testing.

9. Complete Page List

63 pages across 8 groups. Access levels: Public (no auth), Auth (any logged-in user), Role (specific domain role required).

9.1 Public / Auth Pages

Page	Route	Access
Landing page	/	Public
Login	/login	Public
Register / Sign up	/register	Public
Forgot password	/forgot-password	Public
Reset password	/reset-password	Public
Accept invite	/invite/[token]	Public
Error 404	/404	Public
Error 403	/403	Public

9.2 Platform Admin (10 pages)

Page	Route	Access
Platform dashboard	/platform/dashboard	Super admin
Tenant management	/platform/tenants	Super admin
Tenant detail	/platform/tenants/[id]	Super admin
Billing & subscriptions	/platform/billing	Super admin
Global user management	/platform/users	Super admin
LLM & model config	/platform/models	Super admin
Domain registry	/platform/domains	Super admin
Audit log viewer	/platform/audit	Super admin
Usage & analytics	/platform/analytics	Super admin
Platform settings	/platform/settings	Super admin

9.3 Tenant Admin (9 pages)

Page	Route	Access
Tenant dashboard	/[tenant]/dashboard	Tenant admin

Page	Route	Access
Domain management	/[tenant]/domains	Tenant admin
User management	/[tenant]/users	Tenant admin
Role assignment	/[tenant]/users/[id]/roles	Tenant admin
Invitation management	/[tenant]/invitations	Tenant admin
Billing & plan	/[tenant]/billing	Tenant admin
Audit log	/[tenant]/audit	Tenant admin
Tenant settings	/[tenant]/settings	Tenant admin
Notification settings	/[tenant]/settings/notifications	Tenant admin

9.4 Domain Common Pages (per domain)

Page	Route	Access
Domain home	/[tenant]/[domain]	Auth
AI chat	/[tenant]/[domain]/chat	Auth
Chat history	/[tenant]/[domain]/chat/history	Auth
Chat detail	/[tenant]/[domain]/chat/[id]	Auth
Knowledge base	/[tenant]/[domain]/knowledge	Auth
Upload document	/[tenant]/[domain]/knowledge/upload	Knowledge manager
Document detail	/[tenant]/[domain]/knowledge/[id]	Auth
Search	/[tenant]/[domain]/search	Auth
Domain settings	/[tenant]/[domain]/settings	Domain admin
Domain users	/[tenant]/[domain]/users	Domain admin

9.5 HR Domain Specific

Page	Route	Access
HR dashboard	/[tenant]/hr	HR user+
Policies library	/[tenant]/hr/policies	HR user+
Policy detail	/[tenant]/hr/policies/[id]	HR user+
Employee handbook	/[tenant]/hr/handbook	HR user+
Leave & attendance	/[tenant]/hr/leave	HR user+
Recruitment knowledge	/[tenant]/hr/recruitment	HR manager+

9.6 Finance Domain Specific

Page	Route	Access
Finance dashboard	/[tenant]/finance	Finance user+
Invoice library	/[tenant]/finance/invoices	Finance user+
Invoice detail	/[tenant]/finance/invoices/[id]	Finance analyst+
Financial reports	/[tenant]/finance/reports	Finance user+
Expense knowledge	/[tenant]/finance/expenses	Finance user+
Accounting knowledge	/[tenant]/finance/accounting	Finance analyst+

9.7 IT Domain Specific

Page	Route	Access
IT dashboard	/[tenant]/it	IT user+
Technical docs	/[tenant]/it/docs	IT user+
API documentation	/[tenant]/it/docs/api	IT user+
Server documentation	/[tenant]/it/docs/servers	IT engineer+
Incident knowledge	/[tenant]/it/incidents	IT engineer+
Troubleshooting guide	/[tenant]/it/troubleshooting	IT user+

9.8 User Account (4 pages)

Page	Route	Access
Profile	/account/profile	Auth
My domains	/account/domains	Auth
Notification preferences	/account/notifications	Auth
Activity log	/account/activity	Auth

10. Environment Variables

10.1 Next.js (.env.local)

Variable	Dev value	Purpose
NEXTAUTH_URL	http://localhost:3000	NextAuth base URL
NEXTAUTH_SECRET	dev-secret-32chars	Session cookie encryption
NEXT_PUBLIC_API_URL	http://localhost:8000	FastAPI base URL (client-side)
KEYCLOAK_CLIENT_ID	platform-web	Keycloak OIDC client ID
KEYCLOAK_CLIENT_SECRET	dev-secret	Keycloak client secret
KEYCLOAK_ISSUER	http://localhost:8080/realms/platform	Keycloak realm issuer URL
NEXT_PUBLIC_STRIPE_KEY	pk_test_...	Stripe publishable key (client)

10.2 FastAPI (.env)

Variable	Dev value	Purpose
DATABASE_URL	postgresql+asyncpg://user:pass@localhost:5432/platform	PostgreSQL connection
REDIS_URL	redis://localhost:6379/0	Redis cache + Celery broker
S3_ENDPOINT	http://localhost:9000	MinIO endpoint
S3_ACCESS_KEY	minioadmin	MinIO access key
S3_SECRET_KEY	minioadmin	MinIO secret key
S3_BUCKET	platform-docs	Default bucket name
OPENAI_API_KEY	sk-...	OpenAI embeddings + GPT-4o
JWT_SECRET	dev-jwt-secret-64chars	JWT signing secret
JWT_ALGORITHM	HS256	JWT signing algorithm
JWT_EXPIRY_MINUTES	60	Access token expiry
REFRESH_EXPIRY_DAYS	30	Refresh token expiry
KEYCLOAK_URL	http://localhost:8080	Keycloak base URL
KEYCLOAK_REALM	platform	Keycloak realm name

Variable	Dev value	Purpose
STRIPE_SECRET_KEY	sk_test_...	Stripe secret key (server)
STRIPE_WEBHOOK_SECRET	whsec_...	Stripe webhook signature
CLICKHOUSE_URL	http://localhost:8123	ClickHouse audit DB
CLICKHOUSE_DB	platform_audit	ClickHouse database name
SENTRY_DSN	https://...@sentry.io/...	Sentry error tracking DSN
ENVIRONMENT	development	Environment label (dev/staging/prod)

11. Security Architecture

Layer	Mechanism	What it prevents
Network	NGINX + rate limiting	DDoS, brute force, traffic flooding
Authentication	JWT + Keycloak MFA	Unauthorised access, credential theft
Tenant isolation	PostgreSQL RLS on all tables	Cross-tenant data leakage (database-enforced)
Domain isolation	FastAPI RBAC middleware	HR user accessing Finance data
Vector isolation	pgvector queries scoped by tenant_id + domain_id	Cross-tenant vector retrieval
File isolation	MinIO paths: {tenant_id}/{domain_id}/{file_id}	Unauthorised file access
API protection	slowapi rate limits per tenant + Redis	API abuse and scraping
Token security	Single-use invite tokens, short JWT expiry (60 min)	Token replay attacks
Transport	HTTPS everywhere (HSTS in production)	Man-in-the-middle attacks
Secrets	Environment variables only — never in code or git	Secret exposure in repos
Dependencies	Dependabot + Snyk scanning in CI	Supply chain attacks
Audit trail	ClickHouse append-only log — all events	Tampered or missing audit records

⚠ *Never commit .env files to git. Add .env, .env.local, .env.* to .gitignore immediately on project setup.*

💡 *In development, all secrets use obvious placeholder values. Replace every value before any staging or production deployment.*